



Core Concepts

Metrics

Available Metrics

Retrieval Augmented Generation

Context Recall

Copy page

Context Recall measures how many of the relevant documents (or pieces of information) were successfully retrieved. It focuses on not missing important results. Higher recall means fewer relevant documents were left out. In short, recall is about not missing anything important.

Since it is about not missing anything, calculating context recall always requires a reference to compare against. The LLM-based Context Recall metric uses `reference` as a proxy to `reference_contexts`, which makes it easier to use as annotating reference contexts can be very time-consuming. To estimate context recall from the `reference`, the reference is broken down into claims, and each claim is analyzed to determine whether it can be attributed to the retrieved context or not. In an ideal scenario, all claims in the reference answer should be attributable to the retrieved context.

The formula for calculating context recall is as follows:

$$\text{Context Recall} = \frac{\text{Number of claims in the reference supported by the retrieved context}}{\text{Total number of claims in the reference}}$$

Example

```
from openai import AsyncOpenAI
from ragas.llms import llm_factory
from ragas.metrics.collections import ContextRecall

# Setup LLM
```

latest

```
client = AsyncOpenAI()
llm = llm_factory("gpt-4o-mini", client=client)

# Create metric
scorer = ContextRecall(llm=llm)

# Evaluate
result = await scorer.ascore(
    user_input="Where is the Eiffel Tower located?",
    retrieved_contexts=["Paris is the capital of France."],
    reference="The Eiffel Tower is located in Paris."
)
print(f"Context Recall Score: {result.value}")
```

Output:

```
Context Recall Score: 1.0
```

Synchronous Usage

If you prefer synchronous code, you can use the `.score()` method instead of `.ascore()`:

```
result = scorer.score(
    user_input="Where is the Eiffel Tower located?",
    retrieved_contexts=["Paris is the capital of France."],
    reference="The Eiffel Tower is located in Paris."
)
```

  latest ▼

LLM Based Context Recall (Legacy API)

 Legacy API

The following example uses the legacy metrics API pattern. For new projects, we recommend using the collections-based API shown above. This API will be deprecated in version 0.4 and removed in version 1.0.

```
from ragas.dataset_schema import SingleTurnSample
from ragas.metrics import LLMContextRecall

sample = SingleTurnSample(
    user_input="Where is the Eiffel Tower located?",
    response="The Eiffel Tower is located in Paris.",
    reference="The Eiffel Tower is located in Paris.",
    retrieved_contexts=["Paris is the capital of France."],
)

context_recall = LLMContextRecall(llm=evaluator_llm)
await context_recall.single_turn_ascore(sample)
```

Output:

```
1.0
```

Non LLM Based Context Recall

`NonLLMContextRecall` metric is computed using `retrieved_contexts` and `reference_contexts`, and the values range between 0 and 1, with higher values indicating better performance. This metrics uses non-LLM string comparison metrics to identify if a retrieved context is relevant or not.  [latest](#) ▼

The formula for calculating context recall is as follows:

$$\text{context recall} = \frac{|\text{Number of relevant contexts retrieved}|}{|\text{Total number of reference contexts}|}$$

Example

```
from ragas.dataset_schema import SingleTurnSample
from ragas.metrics import NonLLMContextRecall

sample = SingleTurnSample(
    retrieved_contexts=["Paris is the capital of France."],
    reference_contexts=["Paris is the capital of France.", "The Eiffel Tower is one of the most famous landmarks in Paris."]
)

context_recall = NonLLMContextRecall()
await context_recall.single_turn_ascore(sample)
```

Output

```
0.5
```

ID BasedContext Recall

ID Based Context Recall IDBasedContextRecall provides a direct and efficient way to measure recall by comparing the IDs of retrieved contexts with reference context IDs. This metric is particularly useful when you have a unique ID system for your documents and want to evaluate retrieval performance without comparing the actual content.

 [latest](#) ▼

The metric computes recall using retrieved_context_ids and reference_context_ids, with values ranging between 0 and 1. Higher values indicate better performance. It works with both string and integer IDs.

The formula for calculating ID-based context recall is as follows:

$$\text{ID-Based Context Recall} = \frac{\text{Number of reference context IDs found in retrieved context IDs}}{\text{Total number of reference context IDs}}$$

Example

```
from ragas.dataset_schema import SingleTurnSample
from ragas.metrics import IDBasedContextRecall

sample = SingleTurnSample(
    retrieved_context_ids=["doc_1", "doc_2", "doc_3"],
    reference_context_ids=["doc_1", "doc_4", "doc_5", "doc_6"]
)

id_recall = IDBasedContextRecall()
await id_recall.single_turn_ascore(sample)
```

Output

```
0.25
```

 December 11, 2025  December 11, 2025  GitHub     +2

  latest ▼